practical_probability_distributions

June 27, 2026

1 Practical Session: Probability Theory and Distributions

1.1 Simulation, Visualization and Approximation with Python

1.1.1 Goals of the practical session

The objective of this practical session is to connect the theoretical concepts studied in the course with numerical simulation and graphical visualization using Python.

At the end of this session, students should be able to:

- simulate random experiments and estimate probabilities empirically;
- simulate discrete random variables: Bernoulli, Binomial and Poisson;
- simulate continuous random variables: Uniform, Exponential and Normal;
- compute probabilities using PMF, PDF and CDF;
- visualize probability distributions using histograms, bar plots and density curves;
- compare theoretical probabilities with simulated results;
- illustrate approximations between distributions;
- illustrate the Central Limit Theorem using simulations.

1.2 1. Required libraries

[1]:
```
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import bernoulli, binom, poisson, uniform, expon, norm

np.random.seed(123)
```

1.3 2. Simulation of a simple random experiment

We toss a fair coin many times. The theoretical probability of heads is

$$P(H) = 0.5.$$

[2]:
```
n = 10000

tosses = np.random.choice(["H", "T"], size=n)
```

```
number_heads = np.sum(tosses == "H")
empirical_probability = number_heads / n

print("Number of heads:", number_heads)
print("Empirical probability of heads:", empirical_probability)
print("Theoretical probability of heads:", 0.5)
```

```
Number of heads: 5028
Empirical probability of heads: 0.5028
Theoretical probability of heads: 0.5
```

1.4 3. Conditional probability and Bayes' theorem

Let

$$S = \{\text{email is spam}\}$$

and

$$F = \{\text{email is flagged}\}.$$

Assume:

$$P(S) = 0.25, \qquad P(F|S) = 0.92, \qquad P(F|S^c) = 0.08.$$

We estimate (P(S|F)) by simulation and compare it with Bayes' theorem.

[3]:
```
n = 100000

is_spam = np.random.rand(n) < 0.25

is_flagged = np.zeros(n, dtype=bool)
is_flagged[is_spam] = np.random.rand(np.sum(is_spam)) < 0.92
is_flagged[~is_spam] = np.random.rand(np.sum(~is_spam)) < 0.08

empirical = np.sum(is_spam & is_flagged) / np.sum(is_flagged)

P_S = 0.25
P_F_given_S = 0.92
P_F_given_notS = 0.08
P_notS = 1 - P_S

P_F = P_F_given_S * P_S + P_F_given_notS * P_notS
theoretical = (P_F_given_S * P_S) / P_F

print("Empirical P(S|F):", empirical)
print("Theoretical P(S|F):", theoretical)
```

```
Empirical P(S|F): 0.7950712014378543
Theoretical P(S|F): 0.793103448275862
```

1.5 4. Bernoulli distribution

A Bernoulli random variable models one experiment with two possible outcomes:

$$X \sim \mathcal{B}(p).$$

Here we take (p=0.7).

[4]:
```
p = 0.7

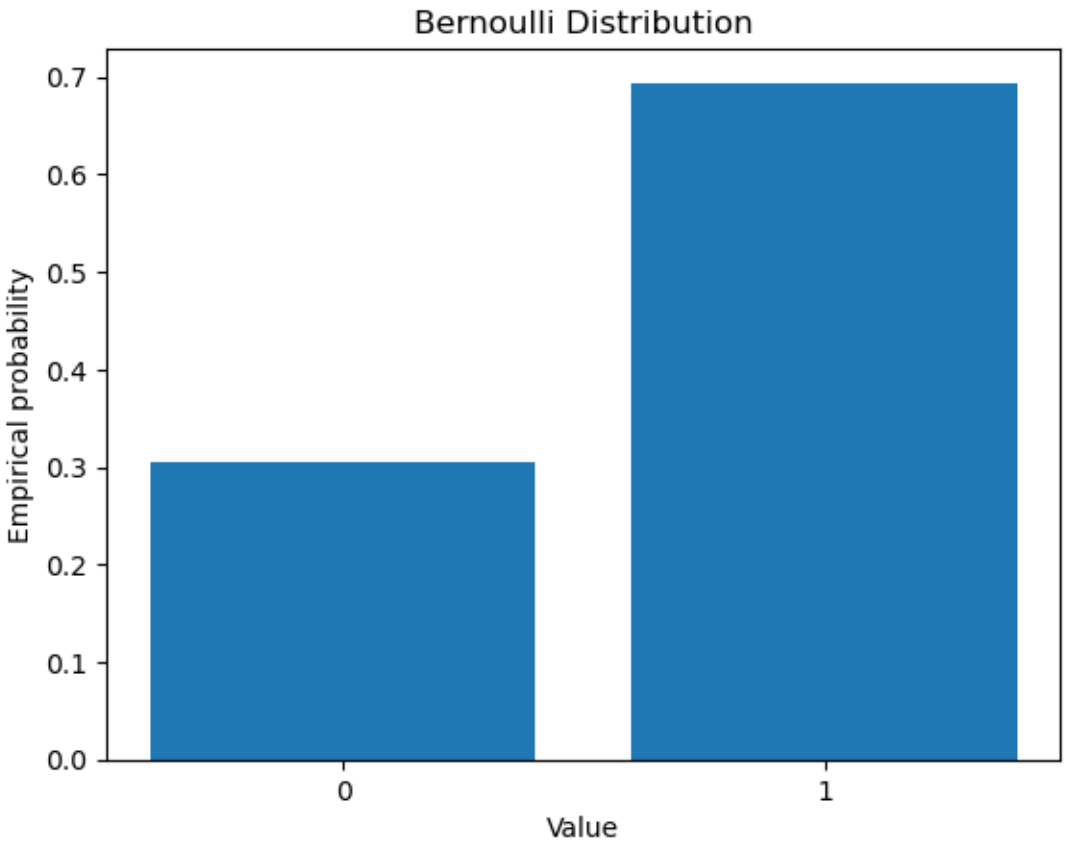
samples = bernoulli.rvs(p, size=1000)

print("Empirical mean:", np.mean(samples))
print("Theoretical mean:", p)

print("Empirical variance:", np.var(samples))
print("Theoretical variance:", p * (1 - p))

plt.bar([0, 1], [np.mean(samples == 0), np.mean(samples == 1)])
plt.xticks([0, 1])
plt.xlabel("Value")
plt.ylabel("Empirical probability")
plt.title("Bernoulli Distribution")
plt.show()
```

```
Empirical mean: 0.694
Theoretical mean: 0.7
Empirical variance: 0.21236400000000002
Theoretical variance: 0.21000000000000002
```



1.6 5. Binomial distribution

A Binomial random variable counts the number of successes in (n) independent Bernoulli trials:

$$X \sim \mathcal{B}(n, p).$$

We simulate

$$X \sim \mathcal{B}(20, 0.4).$$

[5]:
```
n = 20
p = 0.4

samples = binom.rvs(n, p, size=10000)

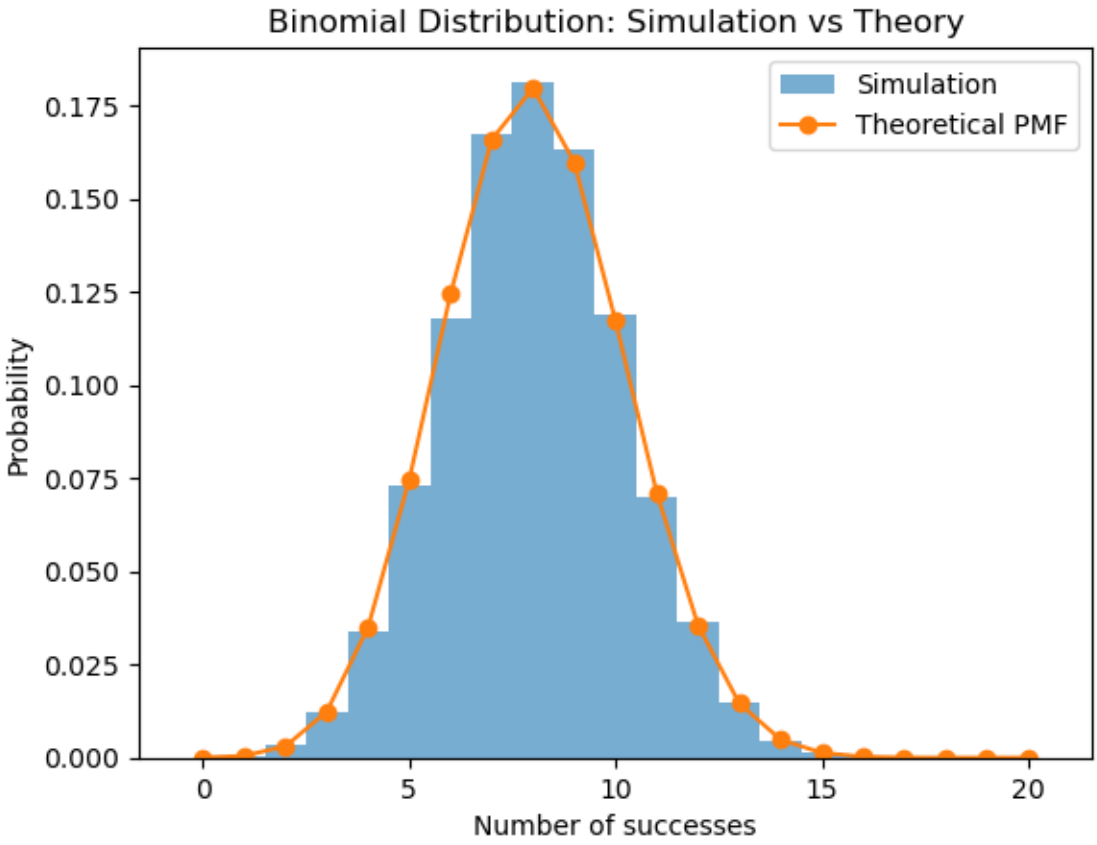
k_values = np.arange(0, n + 1)
theoretical_pmf = binom.pmf(k_values, n, p)

plt.hist(samples, bins=np.arange(-0.5, n + 1.5, 1),
```

```
        density=True, alpha=0.6, label="Simulation")

plt.plot(k_values, theoretical_pmf, "o-", label="Theoretical PMF")

plt.xlabel("Number of successes")
plt.ylabel("Probability")
plt.title("Binomial Distribution: Simulation vs Theory")
plt.legend()
plt.show()
```



1.7 6. Poisson distribution

The Poisson distribution models the number of events occurring during a fixed interval:

$$X \sim \mathcal{P}(\lambda).$$

We simulate

$$X \sim \mathcal{P}(4).$$

[6]:
```
lam = 4

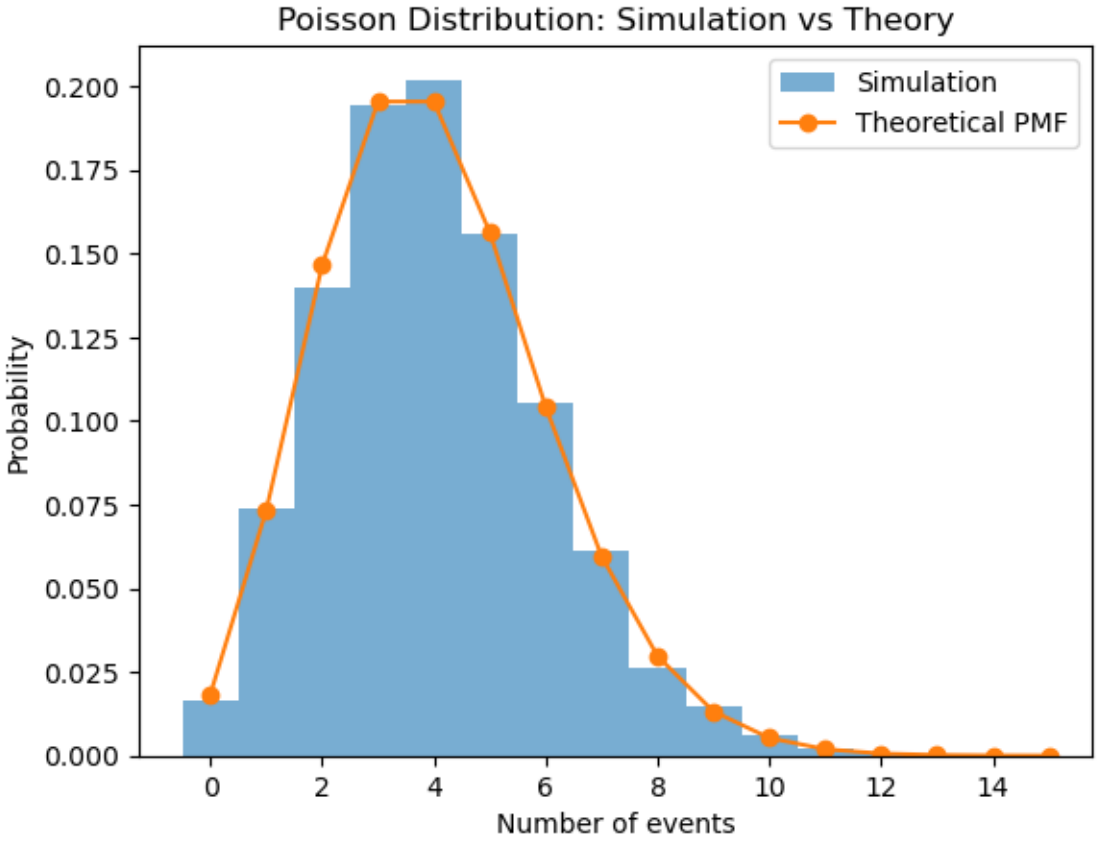
samples = poisson.rvs(lam, size=10000)

k_values = np.arange(0, 16)
theoretical_pmf = poisson.pmf(k_values, lam)

plt.hist(samples, bins=np.arange(-0.5, 15.5, 1),
         density=True, alpha=0.6, label="Simulation")

plt.plot(k_values, theoretical_pmf, "o-", label="Theoretical PMF")

plt.xlabel("Number of events")
plt.ylabel("Probability")
plt.title("Poisson Distribution: Simulation vs Theory")
plt.legend()
plt.show()
```



1.8 7. Poisson approximation of the Binomial distribution

When (n) is large and (p) is small,

$$\mathcal{B}(n, p) \approx \mathcal{P}(np).$$

We consider

$$X \sim \mathcal{B}(100, 0.03).$$

Since (np=3), we approximate it by

$$\mathcal{P}(3).$$

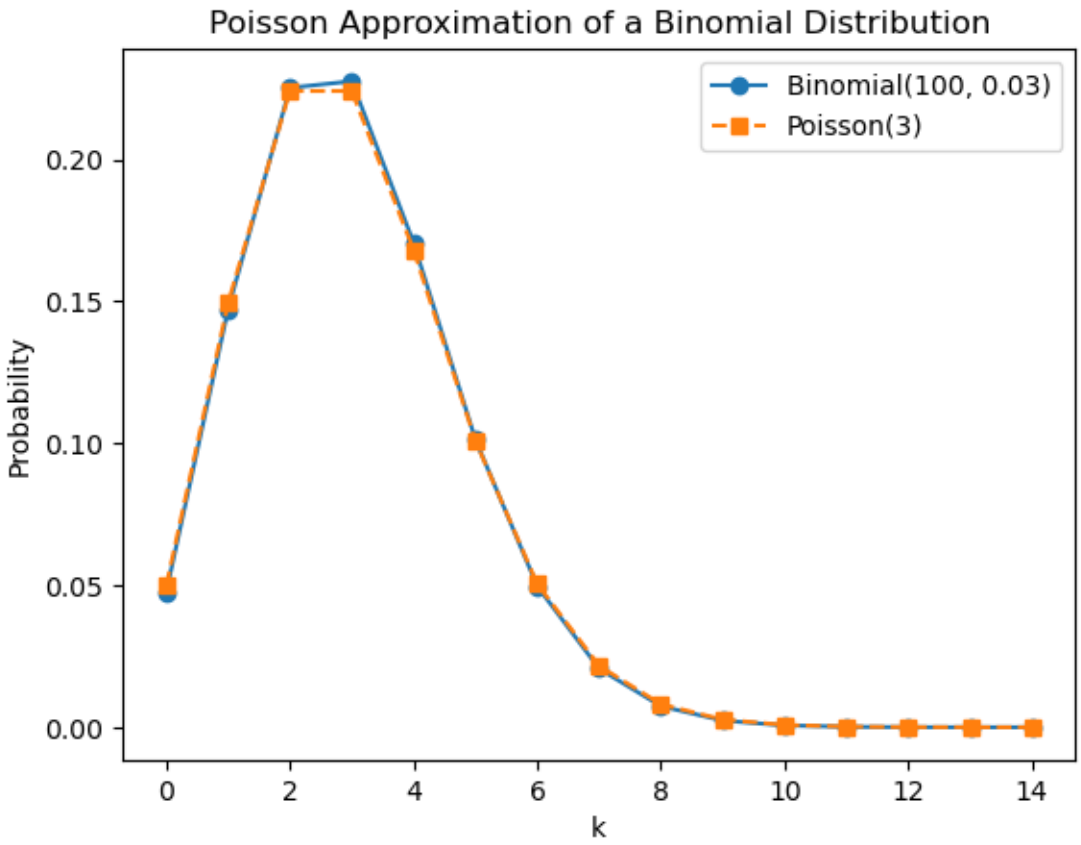
[7]:
```
n = 100
p = 0.03
lam = n * p

k_values = np.arange(0, 15)

binom_pmf = binom.pmf(k_values, n, p)
poisson_pmf = poisson.pmf(k_values, lam)

plt.plot(k_values, binom_pmf, "o-", label="Binomial(100, 0.03)")
plt.plot(k_values, poisson_pmf, "s--", label="Poisson(3)")

plt.xlabel("k")
plt.ylabel("Probability")
plt.title("Poisson Approximation of a Binomial Distribution")
plt.legend()
plt.show()
```



1.9 8. Continuous Uniform distribution

A continuous uniform distribution on ([a,b]) gives the same density to every value inside the interval:

$$X \sim \mathcal{U}(a, b).$$

[8]:
```
a = 0
b = 1

samples = uniform.rvs(loc=a, scale=b-a, size=10000)

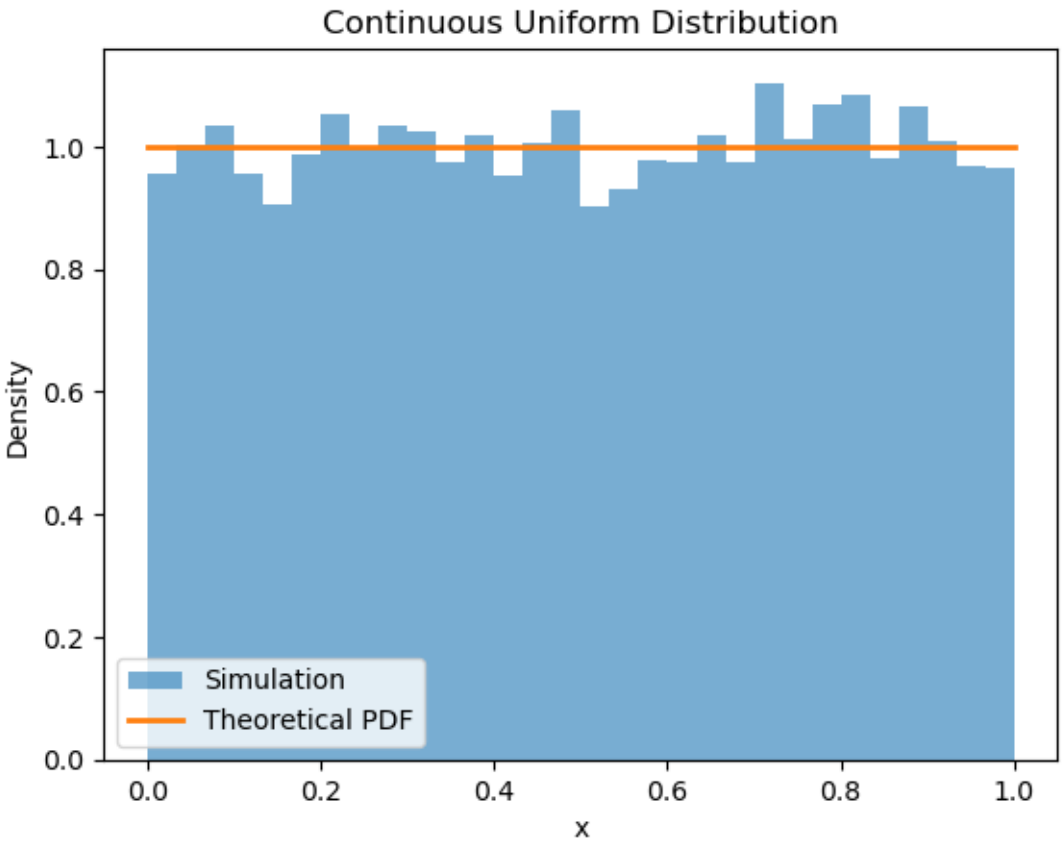
plt.hist(samples, bins=30, density=True, alpha=0.6, label="Simulation")

x = np.linspace(a, b, 200)
pdf = uniform.pdf(x, loc=a, scale=b-a)

plt.plot(x, pdf, linewidth=2, label="Theoretical PDF")

plt.xlabel("x")
```

```
plt.ylabel("Density")
plt.title("Continuous Uniform Distribution")
plt.legend()
plt.show()
```



1.10 9. Exponential distribution

The exponential distribution is often used to model waiting times:

$$X \sim \mathcal{E}(\lambda).$$

In SciPy, the parameter `scale` is equal to

$$\frac{1}{\lambda}.$$

We take (=2).

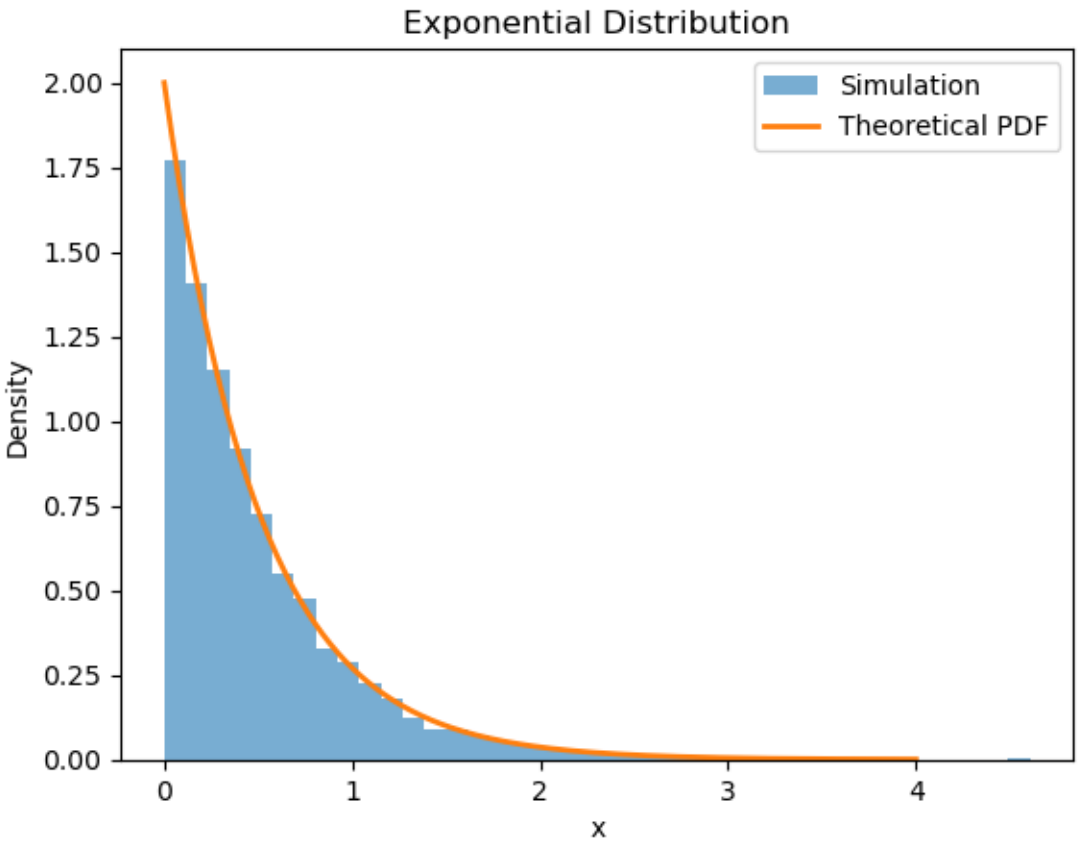
[9]:
```
lam = 2
scale = 1 / lam
```

```
samples = expon.rvs(scale=scale, size=10000)

x = np.linspace(0, 4, 300)
pdf = expon.pdf(x, scale=scale)

plt.hist(samples, bins=40, density=True, alpha=0.6, label="Simulation")
plt.plot(x, pdf, linewidth=2, label="Theoretical PDF")

plt.xlabel("x")
plt.ylabel("Density")
plt.title("Exponential Distribution")
plt.legend()
plt.show()
```



1.11 10. Normal distribution

The normal distribution is one of the most important continuous distributions:

$$X \sim \mathcal{N}(\mu, \sigma^2).$$

We simulate

$$X \sim \mathcal{N}(0, 1).$$

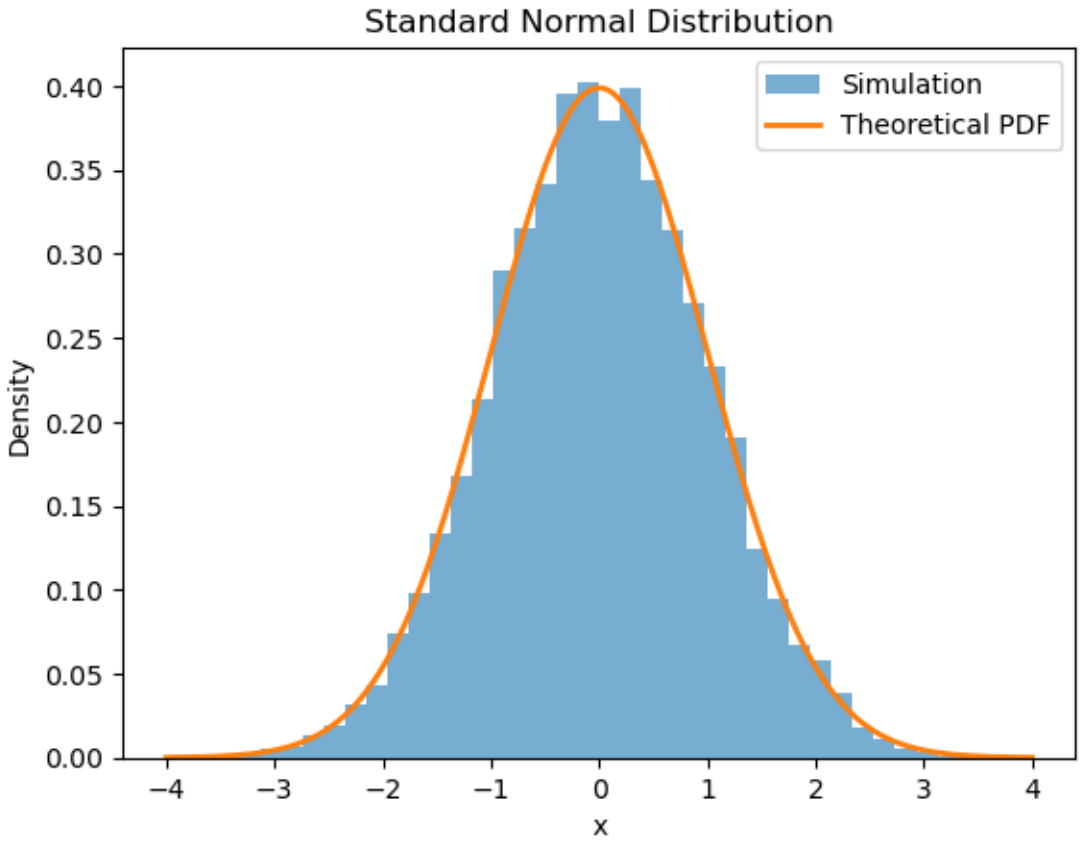
[10]:
```
mu = 0
sigma = 1

samples = norm.rvs(loc=mu, scale=sigma, size=10000)

x = np.linspace(mu - 4*sigma, mu + 4*sigma, 300)
pdf = norm.pdf(x, loc=mu, scale=sigma)

plt.hist(samples, bins=40, density=True, alpha=0.6, label="Simulation")
plt.plot(x, pdf, linewidth=2, label="Theoretical PDF")

plt.xlabel("x")
plt.ylabel("Density")
plt.title("Standard Normal Distribution")
plt.legend()
plt.show()
```



1.12 11. Reading normal probabilities with Python

Let

$$Z \sim \mathcal{N}(0, 1).$$

We compute:

$$P(Z < 1.96), \qquad P(Z > 1.96), \qquad P(-1 < Z < 1).$$

[11]:
```
print("P(Z < 1.96) =", norm.cdf(1.96))
print("P(Z > 1.96) =", 1 - norm.cdf(1.96))
print("P(-1 < Z < 1) =", norm.cdf(1) - norm.cdf(-1))

z_95 = norm.ppf(0.95)
print("z such that P(Z < z) = 0.95:", z_95)
```

```
P(Z < 1.96) = 0.9750021048517795
P(Z > 1.96) = 0.024997895148220484
P(-1 < Z < 1) = 0.6826894921370859
z such that P(Z < z) = 0.95: 1.6448536269514722
```

1.13 12. Normal approximation of the Binomial distribution

When (n) is large and (np(1-p)) is sufficiently large,

$$\mathcal{B}(n, p) \approx \mathcal{N}(np, np(1-p)).$$

We consider

$$X \sim \mathcal{B}(200, 0.2).$$

[12]:
```
n = 200
p = 0.2

mu = n * p
sigma = np.sqrt(n * p * (1 - p))

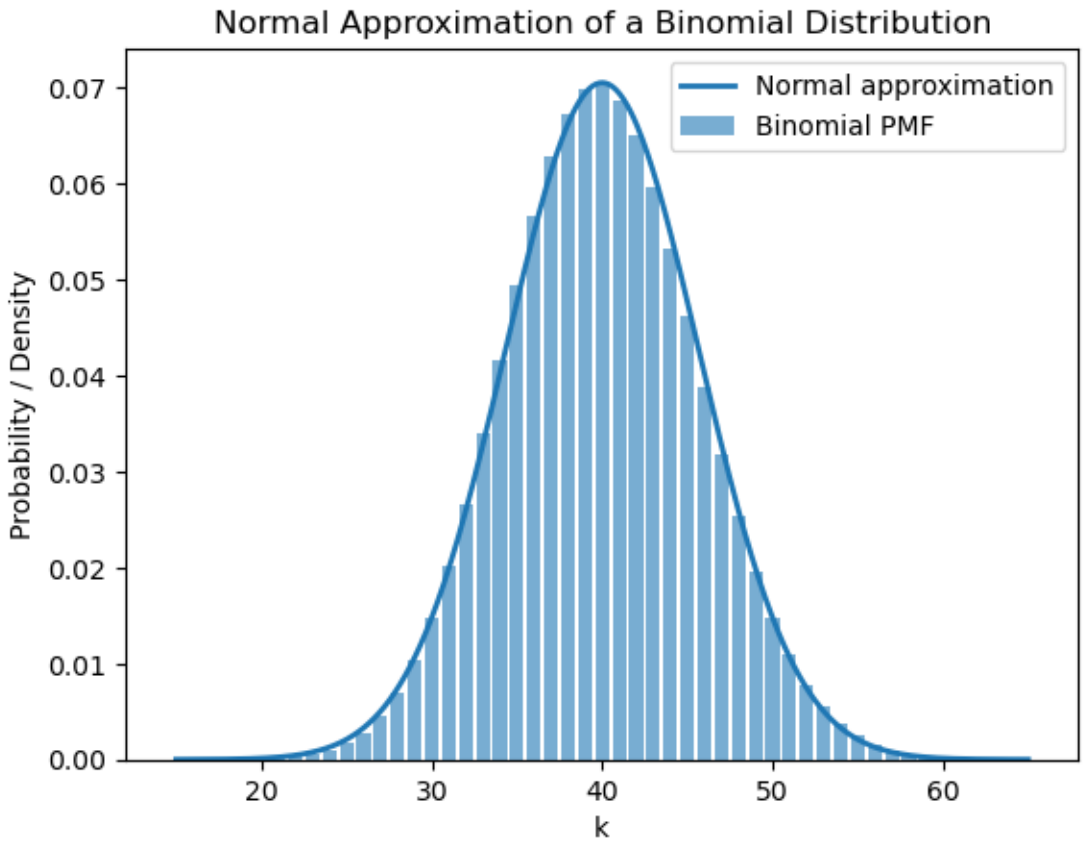
k_values = np.arange(15, 66)
binom_pmf = binom.pmf(k_values, n, p)

x = np.linspace(15, 65, 500)
normal_pdf = norm.pdf(x, loc=mu, scale=sigma)

plt.bar(k_values, binom_pmf, alpha=0.6, label="Binomial PMF")
```

```
plt.plot(x, normal_pdf, linewidth=2, label="Normal approximation")

plt.xlabel("k")
plt.ylabel("Probability / Density")
plt.title("Normal Approximation of a Binomial Distribution")
plt.legend()
plt.show()
```



1.14 13. Continuity correction

When a discrete distribution is approximated by a continuous normal distribution, we use the continuity correction.

For example,

$$P(X > 45) = P(X \geq 46)$$

is approximated by

$$P(Y > 45.5),$$

where (Y) is the normal approximation.

[13]:
```
n = 200
p = 0.2

mu = n * p
sigma = np.sqrt(n * p * (1 - p))

exact_prob = 1 - binom.cdf(45, n, p)
approx_no_correction = 1 - norm.cdf(45, loc=mu, scale=sigma)
approx_with_correction = 1 - norm.cdf(45.5, loc=mu, scale=sigma)

print("Exact P(X > 45):", exact_prob)
print("Normal approximation without correction:", approx_no_correction)
print("Normal approximation with correction:", approx_with_correction)
```

```
Exact P(X > 45): 0.16511752884672393
Normal approximation without correction: 0.18837955890579106
Normal approximation with correction: 0.16545766855695931
```

1.15 14. Normal approximation of the Poisson distribution

If

$$X \sim \mathcal{P}(\lambda)$$

and ( ) is large, then

$$X \approx \mathcal{N}(\lambda, \lambda).$$

We consider

$$X \sim \mathcal{P}(30).$$

[14]:
```
lam = 30

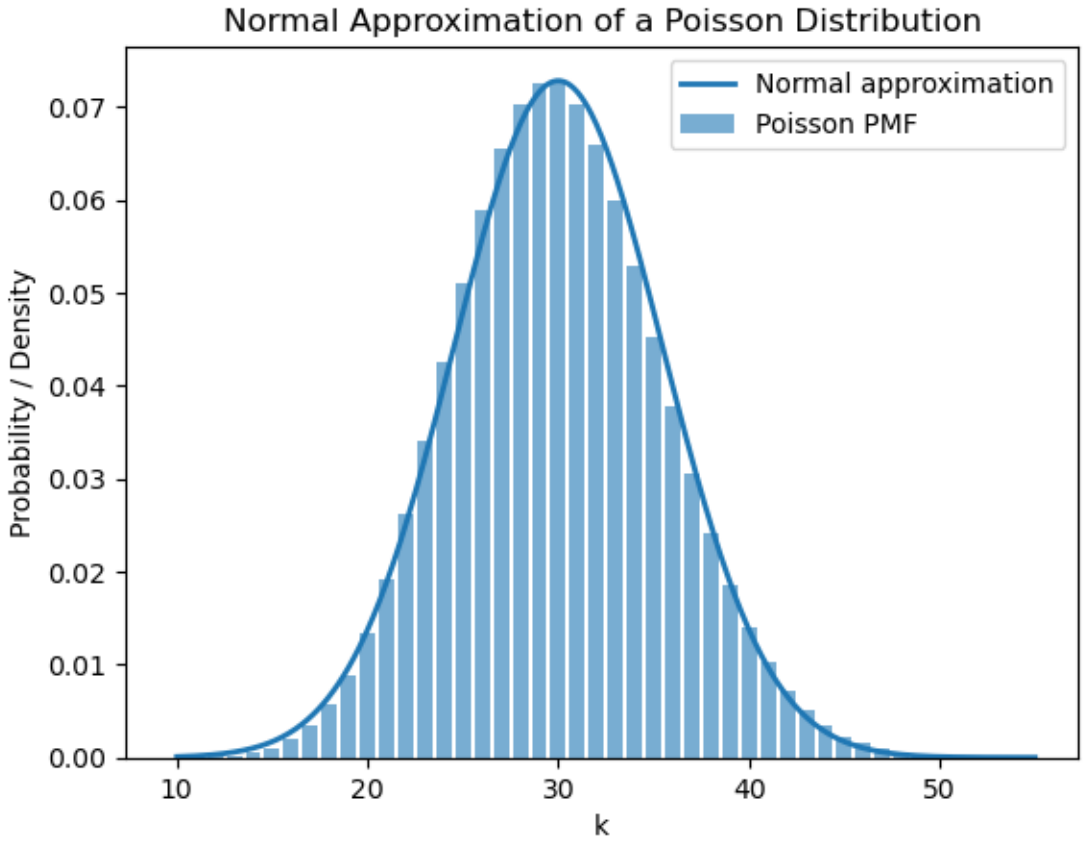
mu = lam
sigma = np.sqrt(lam)

k_values = np.arange(10, 55)
poisson_pmf = poisson.pmf(k_values, lam)

x = np.linspace(10, 55, 500)
normal_pdf = norm.pdf(x, loc=mu, scale=sigma)

plt.bar(k_values, poisson_pmf, alpha=0.6, label="Poisson PMF")
plt.plot(x, normal_pdf, linewidth=2, label="Normal approximation")
```

```
plt.xlabel("k")
plt.ylabel("Probability / Density")
plt.title("Normal Approximation of a Poisson Distribution")
plt.legend()
plt.show()
```



1.16 15. Central Limit Theorem

The Central Limit Theorem states that if (X_1,X_2,…,X_n) are independent and identically distributed random variables with mean ( ) and variance (^2), then for large (n),

$$\overline{X} \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

We illustrate this theorem using an exponential distribution.

[15]:
```
lam = 2
scale = 1 / lam
```

```
sample_size = 40
number_of_samples = 10000

sample_means = []

for i in range(number_of_samples):
    sample = expon.rvs(scale=scale, size=sample_size)
    sample_means.append(np.mean(sample))

sample_means = np.array(sample_means)

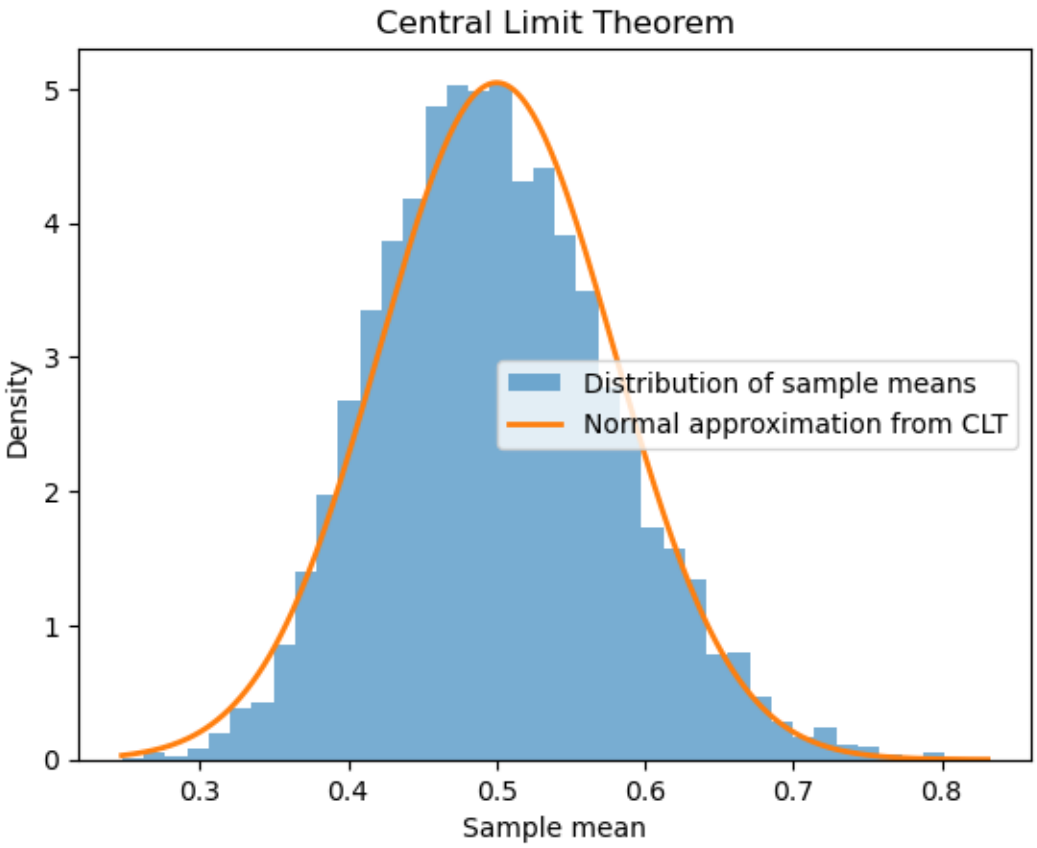
mu = scale
sigma = scale / np.sqrt(sample_size)

x = np.linspace(min(sample_means), max(sample_means), 300)
normal_pdf = norm.pdf(x, loc=mu, scale=sigma)

plt.hist(sample_means, bins=40, density=True, alpha=0.6,
         label="Distribution of sample means")

plt.plot(x, normal_pdf, linewidth=2,
         label="Normal approximation from CLT")

plt.xlabel("Sample mean")
plt.ylabel("Density")
plt.title("Central Limit Theorem")
plt.legend()
plt.show()
```



1.17 16. Tasks for students

1. Change the parameter (p) in the Bernoulli and Binomial simulations. What happens to the shape of the distribution?
2. For the Poisson distribution, try different values of ( ). What happens when ( ) becomes large?
3. In the Poisson approximation of the Binomial distribution, try (p=0.2). Is the approximation still good?
4. In the Normal approximation of the Binomial distribution, compare the approximation with and without continuity correction.
5. In the Central Limit Theorem simulation, replace the exponential distribution by a uniform distribution. Does the distribution of the sample mean still become approximately normal?
6. Try different sample sizes (n=5,10,30,100). What do you observe?

[ ]:

[ ]:

[ ]:

[ ]: